Architectural Improvements for Speech Emotion Recognition Using BiLSTM and Temporal Attention

Itzhak Bista and Gal Ashkenazi

School of Computer Science, Ariel University, Israel

Supervisors Or Haim Anidjar and Roi Yozevitch

Abstract

We investigate whether changes to a compact neural architecture improve speech emotion recognition under a shared experimental protocol. Starting from an existing bidirectional long short-term memory network, we compare four models: the original last-time-step BiLSTM, a convolutional BiLSTM with temporal attention, a BiLSTM with temporal attention, and an attention-based BiLSTM with a multilayer perceptron classifier. All models are trained from random initialization on the 1,440 audio-only speech recordings of RAVDESS, covering eight emotion classes. The experiments use 40-dimensional MFCC inputs, the same stratified recording-level partition, a fixed random seed, Adam optimization, and a budget of 20 epochs. For each architecture, the checkpoint is selected by validation weighted F1 rather than by the last epoch. The BiLSTM with temporal attention achieves the strongest reported test performance: accuracy 0.58, macro F1 0.58, and weighted F1 0.59, compared with 0.40, 0.39, and 0.39 for the baseline. The rounded accuracy gain is 18 percentage points. Adding a convolutional frontend or a larger classifier does not improve upon the simpler attention variant in this run. These results support temporal aggregation as a useful architectural modification for this baseline, while remaining limited to one seed, a recording-level split, and a small acted-speech dataset. We document the implementation, checkpoint policy, class-level errors, reproducibility measures, and limitations of the comparison.

Keywords Speech emotion recognition; BiLSTM; temporal attention; MFCC; RAVDESS; architectural comparison

1 Introduction

Speech emotion recognition assigns an emotion label to an audio recording. The classification decision must use information that changes over time: a short interval may carry a useful cue, while another interval may contribute little to the target label. A recurrent model therefore faces two related problems: representing the sequence and converting that sequence into one utterance-level decision. Our project examines the second problem while retaining a shared recurrent backbone and acoustic input representation.

The central research question is whether replacing the baseline readout with learned temporal aggregation improves recognition, and whether additional convolutional processing or classifier depth offers further benefit. The study focuses on architectural changes within a fixed training pipeline. Its contribution is an implemented and documented comparison, rather than a claim that BiLSTM or attention is a new method.

1 1 Research objectives and contributions

We first reproduce and train the repository baseline in a controlled Colab workflow. We then implement three alternatives by replacing the model definition while preserving the data partition, feature interface, optimizer, learning rate, batch size, training budget, and checkpoint criterion. This design makes the architecture the intended experimental factor. It also makes a negative result useful: a more elaborate network is retained in the comparison even when its score is lower.

The project contributes four concrete outcomes. It establishes a baseline for the selected repository and dataset; implements attention, convolutional, and classifier changes; compares aggregate and class-level performance; and preserves the model definitions, best weights, reports, and a consolidated result table. Reproducibility work supports all four experiments and is reported separately from architectural improvement. The results do not measure the independent benefit of the reproducibility changes.

2 Background and related work

2 1 Acoustic features and sequence modeling

Mel-frequency cepstral coefficients provide a compact representation of an audio signal through coefficients derived from a mel-scaled spectral representation. They are calculated across frames, so an utterance can be represented as a sequence of feature vectors rather than one global average. The librosa MFCC interface exposes the number of coefficients as a configurable parameter [3]. In this project the model input dimension is 40, and the time axis is preserved for recurrent processing.

Long short-term memory networks were introduced to improve learning over extended temporal dependencies using memory cells and multiplicative gates [4]. Our implementation uses a two-layer bidirectional LSTM. The output at each time position concatenates forward and backward representations, yielding 256 values when each direction has hidden size 128. This design supplies contextual representations to the utterance classifier, but the choice of readout still determines how those representations are aggregated.

2 2 Temporal attention and model capacity

Attention-based aggregation assigns learned weights to time positions before forming an utterance representation. Recurrent networks with attention have already been investigated for speech emotion recognition, including the work of Mirsamadi, Barsoum, and Zhang [5]. Our contribution is therefore the evaluation of a particular lightweight attention readout and two extensions within the project baseline. We do not claim to reproduce that paper or compare directly with its reported scores.

The baseline selects one output position. Attention instead calculates a scalar score for each contextual vector and normalizes the scores across time. A weighted sum then feeds the classifier. A convolutional frontend is a separate hypothesis: local operations on neighboring MFCC frames may produce a better sequence for the BiLSTM. A deeper classifier is another hypothesis: nonlinear processing after pooling may better separate emotion categories. These hypotheses are evaluated empirically rather than assumed to improve accuracy.

Cross-paper accuracy comparisons would require matching datasets, emotion inventories, speaker partitions, feature extraction, and evaluation policies. Accordingly, our numerical comparisons are internal to the four saved experiments. The supplied example article guides the manuscript structure, but its ASR methods and results are not part of this study.

3 Dataset and input representation

3 1 RAVDESS speech subset

RAVDESS is a validated collection of emotional speech and song performed by 24 professional actors. Its design includes matched spoken content and several emotional expressions [2]. We use the audio-only speech archive Audio_Speech_Actors_01-24.zip. The notebook downloads and extracts the archive, then confirms that 1,440 WAV files are present. Video and song recordings are outside the experiment.

Labels are read from the third hyphen-separated field of each filename. The mapping is 01 neutral, 02 calm, 03 happy, 04 sad, 05 angry, 06 fearful, 07 disgust, and 08 surprised. LabelEncoder converts the textual categories to integer targets. The classification reports use the order angry, calm, disgust, fearful, happy, neutral, sad, and surprised. Maintaining that order is necessary when interpreting model outputs and confusion matrices.

3 2 Training validation and test partitions

A first stratified train_test_split reserves 20% of the recordings for testing with random_state=42. A second stratified split reserves 20% of the remaining recordings for validation with the same random state. The approximate overall proportions are therefore 64% training, 16% validation, and 20% testing, rather than 60/20/20. Applying the split sizes to 1,440 items gives the counts in Table 1.

Table 1. Recording counts under the implemented two-stage partition.

Partition	Recordings	Purpose
Training	921	Gradient updates
Validation	231	Checkpoint selection
Test	288	Reported evaluation
Total	1,440	Eight emotion classes

The split is stratified by emotion, not grouped by actor. Thus, it does not establish performance on unseen speakers, and recordings from an actor can appear in multiple partitions. Sorting actor folders and WAV paths stabilizes the input order used by the split. Stratification maintains approximate class proportions but does not enforce speaker independence.

3 3 MFCC sequence interface

The dataset and preprocessing components are inherited from the pinned repository [1]. The model accepts tensors with shape batch × time × 40. The notebook checks the modified model interface using a synthetic tensor of shape 4 × 174 × 40 and verifies an output of shape 4 × 8. The value 174 in this check is a test input length; it should not be interpreted as a separately established description of every real recording.

No replacement of the feature extractor is introduced between architectures. However, the notebook does not print the complete dataset and preprocessing source files. Consequently, this manuscript does not assign unverified values to resampling rate, FFT size, hop length, normalization, or padding and truncation settings. Exact reproduction of those inherited details requires the pinned source files. This documentation limitation is distinct from the verified 40-feature model interface.

4 Model architectures

4 1 Original BiLSTM baseline

The starting model is EmotionBiLSTM from the Meghashyam-adimallam speech-emotion-recognition repository [1]. It contains a two-layer bidirectional LSTM with input dimension 40, hidden dimension 128 per direction, and dropout 0.3 between recurrent layers. Its output sequence is $H = (h_1, \ldots, h_t)$, with each h containing 256 components. The implementation selects lstm_out[:, -1, :], applies a linear map from 256 to eight outputs, and returns log-softmax values.

$$z = W\ h_t + b \qquad \log p = \log \mathrm{softmax}(z)$$

Selecting the last output is inexpensive, but it makes the classifier depend on one indexed vector. That vector is not the same as explicitly concatenating the final hidden states of both directions. In a bidirectional layer, the backward component at the last sequence position is an endpoint representation. In a stacked network, contextual information can also reach it through lower-layer outputs, so it would be too strong to say that the model entirely ignores all earlier frames. Nevertheless, the readout does not directly pool the complete output sequence.

4 2 BiLSTM with temporal attention

The attention variant retains the same recurrent configuration and replaces the indexed readout with a learned weighted sum. The TemporalAttention module applies a linear layer from 256 dimensions to one scalar at each time position. The scores are normalized using softmax along the time dimension. The implementation has no additional nonlinear hidden layer within the attention scorer.

$$e_t = w^T h_t + b_a \qquad \alpha_t = \exp(e_t) \,/\, \Sigma_j \exp(e_j)$$

$$c = \Sigma_t\ \alpha_t h_t \qquad z = W_o\ \mathrm{Dropout}(c) + b_o$$

The context vector c has 256 dimensions and is passed through dropout 0.3, a linear 256-to-8 classifier, and log-softmax. The scorer contains 257 stored trainable parameters: 256 weights and one bias. Because the same scalar bias is added to every time score, it cancels in the softmax normalization; the count nevertheless includes it because it is present in the implementation.

The motivation is to let the model learn which contextual time positions are useful for the classification decision. The attention weights are optimized through the emotion classification loss; the notebook does not provide frame-level emotion labels. It also does not visualize the learned weights. Improved recognition is therefore evidence about the prediction architecture, not proof that the model has identified human-interpretable emotional moments.

The baseline-to-attention comparison changes both the pooling operation and dropout placement: an explicit dropout is added before the classifier. Both are architectural choices, but the experiment is not an isolated test of attention alone. A matched dropout control and alternative pooling baselines would be needed to separate their effects.

4 3 CNN with BiLSTM and attention

The convolutional variant places two one-dimensional convolutions before the same BiLSTM and temporal attention module. The input is transposed from batch × time × features to batch × channels × time. Each convolution maps 40 channels to 40 channels, uses kernel size 3 and padding 1, and retains the time length. The first convolution is followed by batch normalization, ReLU, and dropout 0.3. The second is followed by batch normalization and ReLU. The tensor is then transposed back to the recurrent format.

The frontend tests whether local temporal filtering improves the representation supplied to the recurrent layers. Its two kernel-3 convolutions have a nominal five-frame receptive field at stride one before recurrent processing. The BiLSTM is followed by attention, dropout 0.3, a 256-to-8 linear layer, and log-softmax. Batch normalization, nonlinearities, and frontend dropout are part of this architectural intervention, so any difference cannot be attributed to convolution alone.

4 4 BiLSTM with attention and an MLP classifier

The MLP variant preserves the BiLSTM and attention mechanism but replaces the single output layer with dropout, a 256-to-128 linear layer, ReLU, dropout, and a 128-to-8 linear layer. It also returns log-softmax values. The hypothesis is that a nonlinear decision function can better separate pooled utterance representations. Unlike a larger recurrent backbone, this change adds capacity only after temporal aggregation.

All alternatives keep the class name EmotionBiLSTM, allowing the same training script to instantiate whichever architecture is currently written to src/model.py. The notebook reloads the module explicitly for the CNN and attention interface checks. Training runs launch separate Python processes, preventing those checks from substituting a previously imported model for the model on disk.

Table 2. Architecture components and parameter counts calculated from the shown layer definitions. Counts include trainable biases and batch-normalization affine parameters, but exclude running-statistic buffers.

Model	Readout and head	Parameters	Change
Baseline	Last output + linear	571,400	Reference
CNN + attention	Attention + linear	581,497	+10,097
Attention	Attention + linear	571,657	+257
Attention + MLP	Attention + MLP	603,529	+32,129

The parameter calculation uses the PyTorch LSTM layout with four gates and separate input and recurrent bias vectors. For one direction of a layer, the count is $4h(d + h + 2)$, where h is hidden size and d is input size. The two directions use d=40 in the first layer and d=256 in the second. The linear and optional frontend parameters are then added. These are analytical counts from the code, not measurements of latency or memory use.

The attention variant increases the stored parameter count by approximately 0.045% over the baseline. Its favorable test score therefore does not require a materially larger parameter budget. This does not establish a runtime advantage: attention processes all output positions, and neither inference latency nor a controlled memory benchmark is reported.

5 Experimental protocol

5 1 Shared optimization settings

Each architecture is instantiated anew and trained from random initialization. The training script does not load the pretrained checkpoint that happens to be included in the cloned repository. All four experiments use negative log-likelihood loss on the log-softmax outputs and Adam with learning rate 0.001. Adam is an adaptive stochastic optimization method [6]; its use here is inherited from the training setup rather than introduced as an experimental variable.

Table 3. Settings shared by all four training runs.

Setting	Value
Input and outputs	40 MFCC coefficients; 8 emotion classes
Recurrent backbone	2 layers; bidirectional; hidden size 128 per direction
Recurrent dropout	0.3
Optimizer and objective	Adam; learning rate 0.001; NLLLoss
Training budget	20 epochs; batch size 32
Seed	42
Checkpoint criterion	Maximum validation weighted F1
Tie-breaker	Higher validation accuracy
Recorded device	NVIDIA Tesla T4 in Google Colab

The training loader shuffles examples with a separately seeded torch.Generator; validation and test loaders do not shuffle. The notebook does not introduce a learning-rate scheduler, early stopping, or architecture-specific optimizer settings. All 20 epochs are completed even when an earlier checkpoint remains best. Equal settings make the comparison controlled, but do not guarantee that each architecture receives its individually optimal training schedule.

5 2 Validation checkpoint selection

After each epoch, the model enters evaluation mode and predicts labels on the validation set without gradient updates. The code calculates validation accuracy and weighted F1. A checkpoint is replaced when weighted F1 increases by more than 10^{-12}, or when the scores are within that tolerance and accuracy increases. Stale checkpoint files are removed before training. The final epoch does not overwrite a better earlier model.

The selected weights are saved to best_checkpoint.pth and to a compatibility copy named emotion_bilstm.pth. The evaluation script is patched to load best_checkpoint.pth. The checkpoint policy applies to the baseline as well as the modified models. Consequently, the reported comparison is between architectures within this revised shared protocol, not a reproduction of the original repository’s final-epoch scores.

5 3 Metrics

Accuracy is the fraction of correctly classified recordings. For class k, precision measures how many predictions of k are correct, while recall measures how many true examples of k are recovered. F1 is their harmonic mean. Macro F1 weights the eight classes equally; weighted F1 weights each class by its test support. They describe complementary aspects of performance, particularly because neutral has fewer test examples than the other classes.

$$\mathrm{F1}_k = 2\mathrm{P}_k\mathrm{R}_k \,/\, (\mathrm{P}_k + \mathrm{R}_k) \qquad \text{Macro F1} = (1/8)\ \Sigma_k\, \mathrm{F1}_k$$

$$\text{Weighted F1} = \Sigma_k \, (n_k/N) \, F1_k$$

6 Experimental results

6 1 Aggregate performance

Table 4 reports the test metrics printed by the four saved evaluation runs. The notebook comparison code parses these reports rather than entering scores manually. The reports round metrics to two decimal places, and the table preserves that precision. Each evaluation contains 288 test recordings.

Table 4. Test-set results for each validation-selected checkpoint. Differences are computed from the displayed rounded scores.

Model	Accuracy	Macro F1	Weighted F1	Δ accuracy
Baseline	0.40	0.39	0.39	—
CNN + attention	0.51	0.50	0.50	+11 pp
Attention	0.58	0.58	0.59	+18 pp
Attention + MLP	0.56	0.56	0.56	+16 pp

The attention variant ranks first on all three aggregate metrics. Relative to the baseline, its rounded macro F1 gain is 0.19 and its weighted F1 gain is 0.20. The 0.18 absolute accuracy gain corresponds to approximately 45% relative improvement when calculated from 0.40 and 0.58. This relative percentage is descriptive and inherits the rounding of the inputs; the 18-percentage-point difference is the clearer primary comparison.

Both additional variants also outperform the baseline. However, CNN + attention trails attention by 0.07 accuracy and 0.09 weighted F1, while attention + MLP trails by 0.02 accuracy and 0.03 weighted F1. Thus, the experiment does not support a monotonic relationship between architectural complexity and recognition performance.

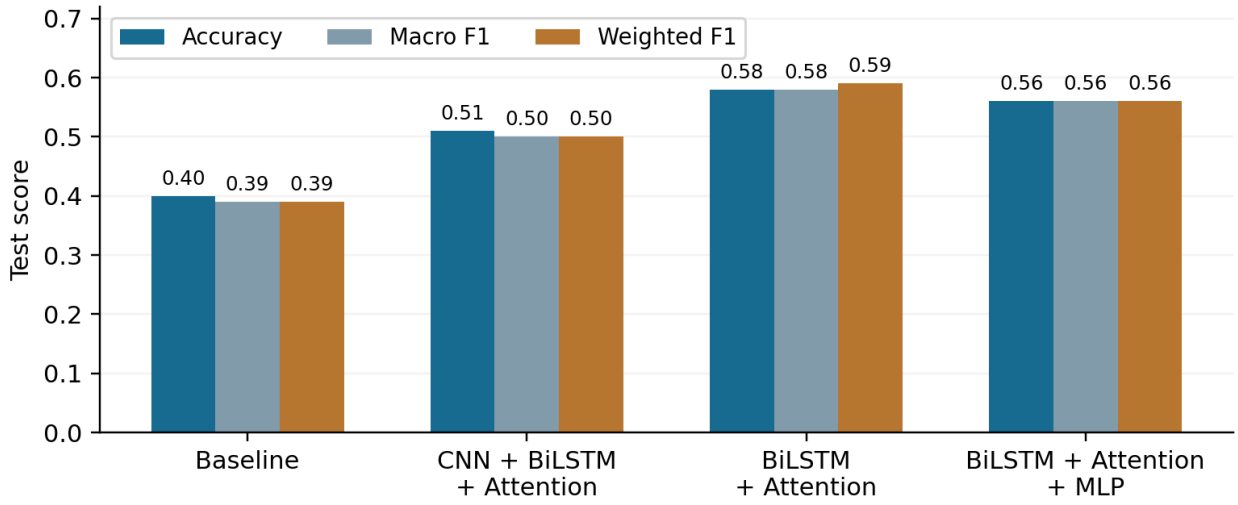


Figure 1. Aggregate test metrics from the saved classification reports. Bars use the reported two-decimal values. No uncertainty bars are shown because only one seed was run.

6 2 What the ranking establishes

The ranking identifies the strongest observed architecture in this experiment. It does not establish statistical significance or superiority across speakers, datasets, or random initializations. Epoch selection uses validation data, but the final architecture ranking uses the test reports. Accordingly, the winning test score is an exploratory comparison result; a fresh held-out evaluation would be needed after locking the architecture for a confirmatory assessment.

6 3 Training behavior and checkpoint epochs

Table 5. Validation metrics at the selected epoch, as printed in the training logs.

Model	Best epoch	Val accuracy	Val weighted F1
Baseline	16	0.4632	0.4454
CNN + attention	17	0.5368	0.5352
Attention	14	0.6190	0.6261
Attention + MLP	18	0.6017	0.6014

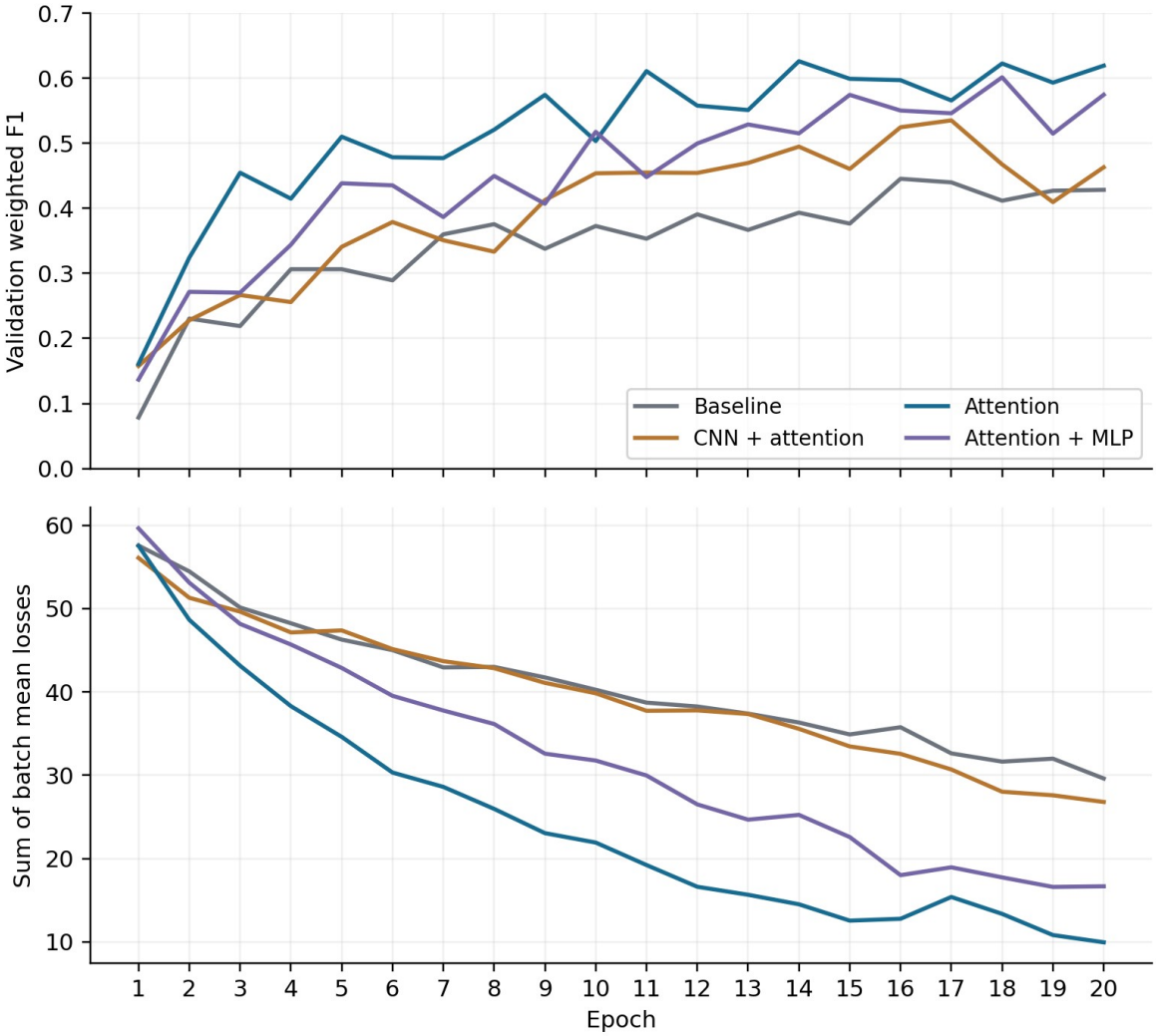


Figure 2. Validation weighted F1 and training loss across all 20 epochs. Curves are reconstructed from saved notebook logs. The loss is the sum of mean batch losses, not the mean loss per recording.

Attention reaches its highest validation weighted F1 at epoch 14 and does not improve upon it in the remaining epochs. Its final-epoch weighted F1 is 0.6193, below the selected 0.6261, despite a lower training loss. The other runs also select earlier checkpoints. This supports retaining the best validation state rather than assuming the final epoch generalizes best.

6 4 Performance by emotion

The aggregate gain is distributed unevenly across classes. Table 6 compares each model's F1 for every emotion using the classification reports. Test support is identical across the four reports. Neutral has 19 examples, whereas the other classes have 38 or 39, so individual neutral errors have a larger effect on its recall.

Table 6. Per-class F1 and test support. A denotes attention and C denotes the convolutional frontend.

Emotion	Support	Baseline	C + A	A	A + MLP
Angry	38	.63	.65	.73	.62
Calm	38	.57	.62	.68	.62
Disgust	38	.46	.54	.58	.54
Fearful	39	.23	.49	.63	.62
Happy	39	.11	.44	.47	.51
Neutral	19	.37	.49	.40	.49
Sad	38	.21	.41	.40	.34
Surprised	39	.52	.39	.71	.73

Compared with the baseline, attention improves F1 in all eight classes at the displayed precision. The largest gains are fearful, from 0.23 to 0.63, and happy, from 0.11 to 0.47. Sad and surprised each improve by 0.19. These results show that the aggregate gain is not explained solely by the already stronger angry and calm classes.

The attention model achieves F1 0.73 for angry, 0.71 for surprised, and 0.68 for calm. Its weakest classes are neutral and sad at 0.40, followed by happy at 0.47. For happy, precision is 0.55 and recall is 0.41, indicating that many true happy examples are missed. For sad, precision is 0.35 and recall is 0.47, indicating substantial false-positive assignment as well as missed sad examples.

The larger classifier provides local gains without winning overall. Attention + MLP improves happy F1 to 0.51, neutral to 0.49, and surprised to 0.73, but reduces angry, calm, disgust, and sad performance relative to attention. The CNN variant also reaches neutral F1 0.49 and slightly exceeds attention on sad, while performing markedly worse on surprised. Model choice based on one class could therefore differ from model choice based on weighted F1.

6 5 Interpretation of optimization curves

All four runs reduce training loss substantially. The baseline falls from 57.5735 to 29.6060; CNN + attention falls from 56.0935 to 26.7884; attention falls from 57.5456 to 9.9370; and attention + MLP falls from 59.6432 to 16.6631. Because the script adds loss.item() across batches, these values should not be interpreted as individual-example negative log-likelihoods.

The divergence between continuing training-loss improvement and fluctuating validation scores is consistent with diminishing generalization gains and possible overfitting. It is not a complete overfitting diagnosis: training accuracy and validation loss were not logged. The results also do not prove that the CNN or MLP variants have plateaued under every possible schedule, since all models share a finite 20-epoch budget.

6 6 Confusion matrix analysis

Figure 3 reproduces the confusion matrix saved for the BiLSTM + Attention experiment before the MLP experiment. Rows are true labels and columns are predictions. The matrix contains 288 observations and 167 diagonal entries, corresponding to accuracy 167/288 = 0.57986, consistent with the reported 0.58.

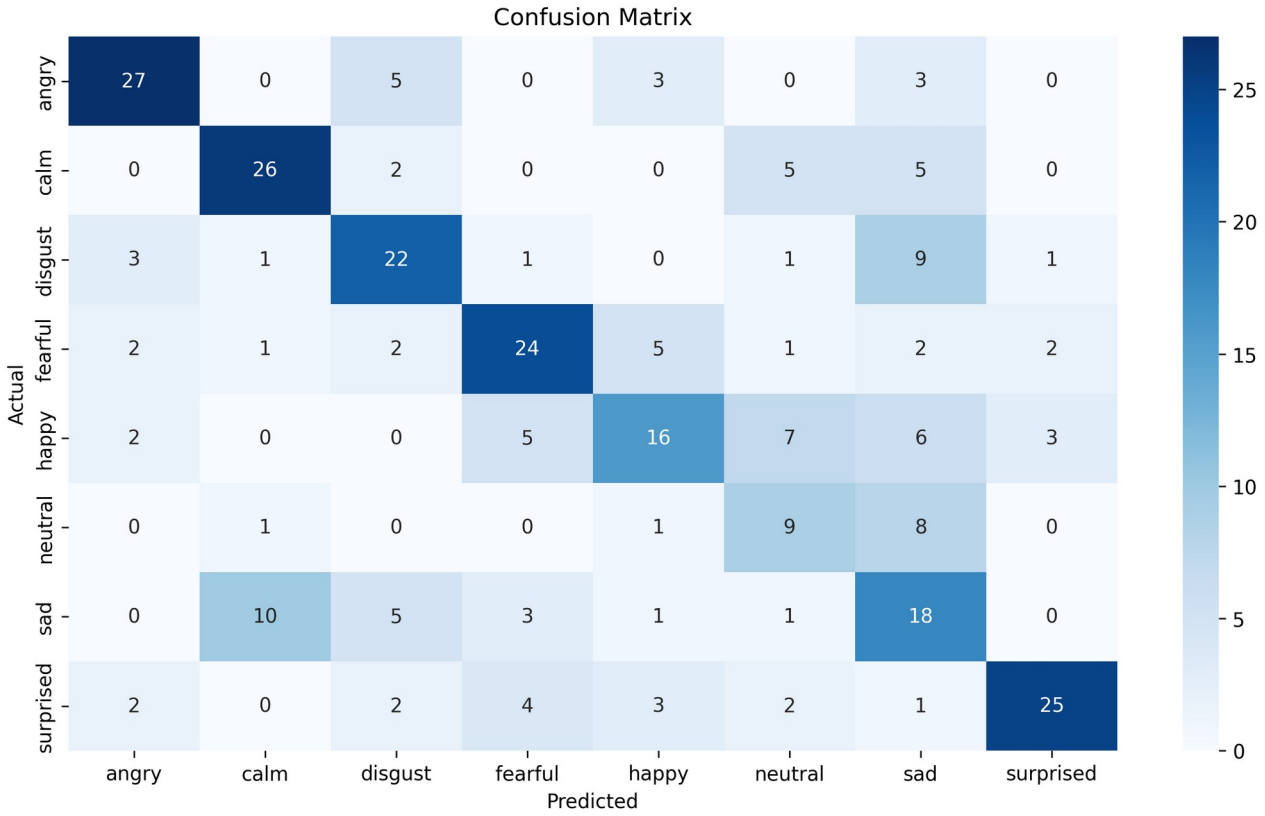


Figure 3. Saved test confusion matrix for BiLSTM + Attention. The figure is taken from the notebook output, not reconstructed from rounded F1 values.

The model correctly classifies 27 of 38 angry recordings, 26 of 38 calm recordings, 22 of 38 disgust recordings, 24 of 39 fearful recordings, and 25 of 39 surprised recordings. Happy, neutral, and sad are more difficult, with 16/39, 9/19, and 18/38 correct, respectively.

Several errors concentrate around the sad prediction. Eight of 19 neutral examples are predicted as sad, nine of 38 disgust examples are predicted as sad, and six of 39 happy examples are predicted as sad. Conversely, ten of 38 sad examples are predicted as calm. Happy recordings also produce seven neutral predictions and five fearful predictions. These counts explain why overall improvement coexists with relatively weak sad and neutral F1.

The matrix supports identifying the affected label pairs, but it does not establish their acoustic cause. Explanations involving similar prosody, recording duration, padding, or actor-specific characteristics remain hypotheses until the corresponding recordings and intermediate representations are analyzed. We therefore avoid interpreting a confusion pattern as evidence of a particular psychological or acoustic mechanism.

7 Discussion of architectural improvements

7 1 What changed relative to the starting model

The principal successful change replaces a single-position readout with a learned aggregation of the BiLSTM output sequence. This gives the classifier direct access to a weighted combination of contextual representations at all time positions. It preserves the feature dimension, recurrent depth, hidden size, and output inventory. The resulting accuracy increase from 0.40 to 0.58 is accompanied by gains in both macro and weighted F1, making it broader than a change in one aggregate score.

The attention model has almost the same parameter count as the baseline. This is compatible with the interpretation that the readout design matters more than simply increasing capacity for this particular setup. However, the attention variant also adds classifier-input dropout, and its different module initialization consumes a different random-number sequence. A shared seed makes each run repeatable; it does not imply identical initialization of every comparable tensor or isolate the effect of one mathematical operation.

7 2 Why added components may not help

The CNN frontend changes the feature sequence before recurrent processing and introduces batch normalization. Its lower score could reflect an unfavorable inductive bias for these MFCC inputs, optimization difficulty under the shared schedule, or sensitivity to limited training data. These are possible explanations, not measured causes. The observed result is narrower: this particular frontend, trained with these settings, scores below attention without the frontend.

The MLP creates a more expressive classifier but adds roughly 32 thousand parameters relative to the baseline. The current data and training budget may not make that extra capacity useful for aggregate generalization. Nevertheless, its improvements on happy, neutral, and surprised show that it changes the class trade-offs rather than uniformly degrading predictions. Architecture-level evaluation should therefore retain both the overall ranking and the per-class evidence.

7 3 Limits of the causal interpretation

The common training script, fixed partition, and uniform checkpoint policy reduce variation unrelated to architecture. They do not make the study a complete component ablation. There is no baseline with matched output dropout, no mean- or max-pooling control, no CNN model without attention, and no alternative final-hidden-state readout. The experiment supports the implemented attention package relative to the implemented baseline; it does not quantify the isolated causal contribution of every layer.

A clean extension would introduce these controls while keeping the same data manifests and selecting architecture decisions on validation data. In particular, mean pooling and a correctly concatenated pair of terminal hidden states would help determine whether learned weighting is essential or whether a stronger general sequence summary accounts for much of the gain. These are proposed experiments, not completed results.

8 Reproducibility and implementation record

The notebook pins the cloned repository to the latest commit before its specified baseline cutoff and prints the resolved commit 1de5c0c393843dfabd0c70b52cfeffa0d70015fd. Python, NumPy, and PyTorch random generators are seeded with 42; CUDA generators are seeded when available. Actor folders and WAV paths are sorted. The training loader receives its own seeded generator. Deterministic algorithms are enabled, cuDNN benchmarking is disabled, and CUBLAS_WORKSPACE_CONFIG is set to :16:8.

An environment file stores selected installed dependency versions for later reuse. The recorded run uses Python 3.13.15, PyTorch 2.11.0+cu128, CUDA 12.8, NumPy 2.1.3, librosa 0.11.0, and scikit-learn 1.6.1 on a Tesla T4. These settings document the observed environment; they do not guarantee bitwise identical results across different hardware or library builds. The lock file covers selected direct dependencies rather than the entire transitive environment.

The notebook saves separate experiment folders for the baseline, CNN + attention, attention, and attention + MLP. Each includes a model definition, selected weights, and an evaluation report. The attention folder also preserves its confusion-matrix PNG. The final comparison table is exported to model_comparison_results.csv, and the highest-ranked experiment is copied to final_best_model. The final model therefore follows the calculated ranking rather than being fixed in advance to the most complex architecture.

One artifact detail matters when revisiting the results: evaluate.py is patched to write confusion_matrix_attention.png during the attention experiment, and the later MLP evaluation still uses that filename. The working-directory image can therefore be overwritten by the MLP run. The attention image already displayed in the notebook and copied into its experiment folder remains the appropriate source for Figure 3. Unique filenames would remove this ambiguity in future runs.

9 Limitations and future work

The main limitation is the recording-level split. Without actor-grouped partitions, the results cannot be described as speaker-independent recognition. A next evaluation should separate actors across training, validation, and test sets and retain a final test partition that is not used to choose among architectures. This would change the evaluation question and might change the ranking; no such results are claimed here.

Only one seed and one dataset were evaluated. Repeated seeds would allow reporting variation, while additional corpora or cross-corpus tests would examine transfer beyond the RAVDESS recording conditions. The current 288-item test set is small, especially for the 19 neutral examples. No confidence intervals, paired significance tests, or robustness experiments were performed. The observed differences should not be presented as statistically established general improvements.

Future work should also archive the complete preprocessing and evaluation sources, explicit split manifests, full-precision metrics, predictions, and all training histories. Length-aware pooling or masking should be evaluated if inherited preprocessing pads sequences, since the shown attention code has no mask input. Class-balanced objectives, alternative pooling controls, and runtime measurements are further testable directions. They are not part of the completed experiment.

10 Conclusions

We compared an original BiLSTM speech emotion classifier with three architectural modifications using the same RAVDESS subset and shared training protocol. The strongest observed model was BiLSTM with temporal attention, which achieved test accuracy 0.58, macro F1 0.58, and weighted F1 0.59. The baseline achieved 0.40, 0.39, and 0.39, respectively. The attention variant improved every class-level F1 at the reported precision while adding only 257 stored trainable parameters.

The CNN frontend and deeper MLP classifier improved upon the baseline but did not surpass the simpler attention variant. The project therefore provides evidence that the way recurrent outputs are summarized can be more useful than adding extra components under this fixed experimental budget. The strongest defensible conclusion is specific to the saved experiment: the implemented attention readout is a promising improvement to this baseline. Speaker-independent testing, repeated runs, and tighter component controls remain necessary before making broader claims.

Data and code availability

The experiment notebook [7] contains the model definitions, training patches, logs, classification reports, and saved attention confusion matrix used in this manuscript. The baseline repository [1] and RAVDESS source [2] identify the inherited implementation and dataset. Project artifacts are saved under SER_Project in the authors' Google Drive; no independent public release of the trained weights is asserted.

References

[1] Meghashyam-adimallam. Speech emotion recognition. GitHub repository. Baseline commit 1de5c0c393843dfabd0c70b52cfeffa0d70015fd, as recorded in the project notebook. Pinned source repository

[2] Livingstone, S. R., and Russo, F. A. (2018). The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A dynamic, multimodal set of facial and vocal expressions in North American English. PLOS ONE, 13(5), e0196391. doi:10.1371/journal.pone.0196391

[3] librosa development team. librosa.feature.mfcc. Version 0.11.0 documentation. MFCC documentation

[4] Hochreiter, S., and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780. Author-hosted paper

[5] Mirsamadi, S., Barsoum, E., and Zhang, C. (2017). Automatic speech emotion recognition using recurrent neural networks with local attention. Proceedings of ICASSP, 2227–2231. doi:10.1109/ICASSP.2017.7952552

[6] Kingma, D. P., and Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv:1412.6980. Original paper

[7] Bista, I., and Ashkenazi, G. SER project Colab notebook, Untitled0.ipynb. Saved experimental outputs inspected on 14 September 2026. Project notebook